

M12 / STUDY COMPANION

M12 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first

4

Use after learning, not instead of learning

4

DE12-01 | Why Spark exists: driver, executors, partitions and tasks

4

Core recall

4

DE12-02 | PySpark setup, SparkSession and laptop-safe local mode

4

Core recall

4

DE12-03 | DataFrames and explicit schemas

5

Core recall

5

DE12-04 | Spark SQL and expression equivalence

5

Core recall

5

DE12-05 | Lazy transformations and actions

6

Core recall

6

DE12-06 | Jobs, stages, tasks and physical plans

7

Core recall

7

DE12-07 | repartition vs coalesce

7

Core recall

7

DE12-08 | Aggregations and shuffle cost

8

Core recall

8

DE12-09 | Joins and broadcast strategy

8

Core recall

8

DE12-10 | Join correctness, fan-out and reconciliation

9

Core recall 9

DE12-11 | Data skew 9

Core recall 9

DE12-12 | Adaptive Query Execution (AQE) 10

Core recall 10

DE12-13 | Caching and persistence 10

Core recall 10

DE12-14 | Parquet I/O and partition pruning 11

Core recall 11

DE12-15 | Built-in functions vs Python UDFs 11

Core recall 11

DE12-16 | Driver safety: collect, toPandas and bounded inspection 12

Core recall 12

DE12-17 | Testing and debugging Spark transformations 13

Core recall 13

DE12-18 | Integrated PySpark pipeline and distributed-processing checkpoint 13

Core recall 13

Read this first

Use after learning, not instead of learning

Read one card, close this book, reconstruct the model/plan/control from memory, then use the changed prompt. Weak recall routes to that lesson only.

DE12-01 | Why Spark exists: driver, executors, partitions and tasks

Core recall

Use the execution model to explain evidence, not as vocabulary theatre. For every Spark incident be able to state where the code runs, which operation created the partition boundary, what one output row means and which row/key/total check proves business correctness.

Most dangerous wrong turn

Do not say 'a partition is a server' or 'one row equals one task.' One executor can process many partitions over time, and one partition can contain many rows. Also do not infer a production cluster from local[2]. Local mode is a useful execution engine on one machine, not distributed hardware.

Retrieval prompt

A stage has 80 partitions and the cluster has 8 executor cores available. Explain why 80 tasks do not run simultaneously and why reducing to 8 partitions automatically is not always the right repair.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Driver, executors, partitions, distributed work mental model [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark 4.2.0 DataFrame quickstart Structured distributed API orientation [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/quickstart_df.html

DE12-02 | PySpark setup, SparkSession and laptop-safe local mode

Core recall

A trustworthy runtime record says what was really executed on which environment. Never turn static code inspection into a live-Spark PASS. Stable setup and honest runtime evidence make later performance observations comparable.

Most dangerous wrong turn

Common setup mistakes are mixing system Python and venv Python, installing multiple Spark versions, assuming Java 8/11 guidance from an old video is current, or opening Airflow/Docker/PostgreSQL/Spark together on an 8 GB laptop. Another mistake is treating pip install pyspark success as proof that a Spark job actually starts.

Retrieval prompt

Suppose a tutorial uses local[*] and an 8g driver. Explain why blindly copying it onto your laptop could make the lesson less reliable, and how you would choose a safer teaching configuration without pretending it is production tuning.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Local PySpark setup and first SparkSession context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: PySpark 4.2.0 installation Python 3.10+; Java 17+; pip/venv options [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/install.html

DE12-03 | DataFrames and explicit schemas

Core recall

Prefer explicit schemas at stable ingestion boundaries and built-in expressions for transformations. Save printSchema() evidence, row grain and a small bounded sample; the schema alone is not proof that all business values are valid.

Most dangerous wrong turn

Do not assume explicit schema means bad values are automatically quarantined with useful reasons; input error behavior still needs a contract. Do not mark every field nullable just to avoid errors. And do not confuse schema with grain: a perfect schema cannot tell you whether one row is one order, item or event unless you state it.

Retrieval prompt

The partner adds a nullable carrier_code column. Explain how you would update the schema deliberately and how you would distinguish compatible additive change from an unexpected rename of customer_id.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) DataFrame reading and schema-oriented workflow [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: PySpark DataFrame API Structured DataFrame operations and schema surface [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/dataframe.html>

DE12-04 | Spark SQL and expression equivalence

Core recall

Be bilingual enough to review both APIs, but validate business equivalence with row counts, keys and totals. Matching screenshots are weak evidence; deterministic comparison of values and schema is stronger.

Most dangerous wrong turn

A common false comparison is `df.show()` versus a SQL query with `ORDER BY` and assuming row-order difference means value mismatch. DataFrames are not guaranteed ordered unless you request ordering. Another mistake is registering a view before adding `gross_sales`, then writing SQL that assumes the column already exists.

Retrieval prompt

Rewrite a DataFrame filter for orders on/after 2026-09-03 into SQL. Then explain which part is business logic and which part only controls output ordering.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

DataFrame/Spark SQL transformation examples [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Spark DataFrame quickstart Temporary views and SQL/DataFrame examples [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/quickstart_df.html

DE12-05 | Lazy transformations and actions

Core recall

Plan first, trigger work deliberately, and record which actions exist. Do not equate Python line count with Spark work. Use `explain`, Spark UI/runtime metrics and independent result controls together.

Most dangerous wrong turn

Do not say 'nothing happens' before an action in an absolute sense: analysis/planning and metadata work can occur, and some operations may interact with catalog/source metadata. The important boundary is that full data computation is deferred. Also do not call `count()` casually after every debugging line; each action can trigger real work.

Retrieval prompt

A teammate adds `df.count()` only to print 'loaded successfully' before every stage. Explain when that check could be useful and when it is redundant expensive work. Propose a more targeted invariant.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Transformation chain and action context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark DataFrame quickstart Lazy DataFrame transformations and actions context [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/quickstart_df.html

DE12-06 | Jobs, stages, tasks and physical plans

Core recall

Use plan evidence as a map, not fortune-telling. Pair `explain('formatted')` with actual runtime metrics and result reconciliation. A faster wrong plan is still wrong.

Most dangerous wrong turn

Do not label every operation a shuffle. Simple select/filter/withColumn can often remain narrow. Conversely, do not assume a groupBy is cheap because the result has only four rows; many source rows may have moved before those four results appeared. Small output is not small work.

Retrieval prompt

A query adds `orderBy(gross_sales desc)` after groupBy. Explain why ordering can add another redistribution/sort requirement and why the exact stage count should be observed rather than guessed.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Execution-plan and distributed-stage context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: DataFrame.explain Inspect logical/physical plans instead of guessing [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.explain.html>

DE12-07 | repartition vs coalesce

Core recall

Change partitioning for a stated reason, inspect the plan/runtime effect, and revalidate rows/keys/totals. Physical tuning must not silently change business semantics.

Most dangerous wrong turn

`repartition(1)` before every write is a classic anti-pattern: it serializes output through one partition and can overload one task. The opposite mistake is forcing thousands of partitions because 'more parallelism is faster.' On an 18-row lab both choices can look harmless; production evidence is what matters.

Retrieval prompt

After a strong filter leaves 2% of rows, the DataFrame still has hundreds of partitions. Compare coalesce versus repartition for the final write and state when the cheaper narrow reduction is attractive.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Partition-count changes in practical Spark work [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: DataFrame.repartition Hash repartitioning and partition count [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.repartition.html>

DE12-08 | Aggregations and shuffle cost

Core recall

Do not fear shuffle; make it intentional. Reduce irrelevant data early, preserve the requested grain, inspect Exchange/runtime evidence and reconcile totals after every distributed aggregation.

Most dangerous wrong turn

Two dangerous shortcuts are increasing shuffle partitions blindly and collecting raw rows to aggregate in Python. The first may add overhead; the second defeats distributed execution and risks driver memory. Another error is changing the aggregation grain while 'optimizing', such as grouping by region+category when the requested output is region only.

Retrieval prompt

The business asks for top 100 customers by sales. Explain why ``groupBy customer -> sum -> orderBy desc -> limit 100`` can involve more than one expensive distributed boundary and what you would inspect.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) `groupBy/shuffle` context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark SQL performance tuning Partition tuning and shuffle-aware optimization [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

DE12-09 | Joins and broadcast strategy

Core recall

Validate join grain and key uniqueness before tuning. Treat broadcast as a strategy backed by size/plan evidence, not a rule attached to dimension tables.

Most dangerous wrong turn

Never broadcast because a table is called `dim_customer`. Check size. Never use a broadcast hint to hide a many-to-many join. And do not assume the same join strategy is optimal forever: a table can grow, stats can become stale and AQE can adapt decisions at runtime.

Retrieval prompt

The lookup becomes 60 MB compressed on disk but 300 MB in memory. Explain why file size alone is a poor basis for a broadcast decision and which Spark statistics/runtime evidence you would inspect.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Join strategy context [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Spark SQL performance tuning Automatic broadcast, join hints and runtime strategy [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

DE12-10 | Join correctness, fan-out and reconciliation

Core recall

Carry relational discipline into Spark. Every important join should have declared left/right grain, expected cardinality, unmatched policy, key checks and at least one row/key/measure reconciliation.

Most dangerous wrong turn

Do not use `dropDuplicates()` after the join as a universal fix: which contact row survives can be arbitrary and you may discard legitimate facts. Do not use `sum(distinct gross_sales)` either; two genuine orders can have the same amount. Repair at the grain/key rule, not at the appearance of duplicated values.

Retrieval prompt

A product dimension unexpectedly has two active rows for P09. Describe a pre-join uniqueness test and a post-join count/amount reconciliation that would stop the pipeline before serving wrong totals.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Join mechanics; book adds grain/fan-out reconciliation [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: DataFrame API Join is an API operation; correctness still depends on keys/cardinality [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/dataframe.html>

DE12-11 | Data skew

Core recall

Diagnose skew from distributions and runtime metrics. Choose a remedy that preserves the aggregation/join semantics, and prove both performance change and unchanged business totals.

Most dangerous wrong turn

Random repartition without a key can redistribute rows for one stage but the next `groupBy/join` on HOT must bring the key back together. Salting without a second-stage recombination changes the result. And treating the longest task as skew without checking data distribution can confuse slow I/O or GC with hot keys.

Retrieval prompt

The HOT key is required to be processed as one logical group for a non-associative operation. Explain why naive salting may be invalid and what architecture alternatives you would consider.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Uneven partitions and skew context [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Spark SQL performance tuning Skew handling and AQE skew optimization [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

DE12-12 | Adaptive Query Execution (AQE)

Core recall

Keep AQE enabled unless you have evidence-based reasons otherwise, but treat it as one layer of execution optimization. Validate data distribution, join correctness and output invariants independently.

Most dangerous wrong turn

Do not say AQE 'automatically makes Spark fast.' It can add planning benefits only where supported and may leave root causes untouched. Do not compare elapsed time from one tiny local run as proof. Warm caches, JVM startup and background load can dominate classroom timings.

Retrieval prompt

Suppose AQE coalesces many tiny post-shuffle partitions but one hot key still dominates a task. Explain why both facts can be true and what next evidence you need.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Adaptive execution context [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Spark SQL performance tuning AQE, coalescing post-shuffle partitions and adaptive joins [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

DE12-13 | Caching and persistence

Core recall

Cache intentionally, materialize knowingly, inspect actual reuse/storage, and unpersist when the reuse window ends. Never use cache as a reflexive line in every pipeline.

Most dangerous wrong turn

Caching every intermediate DataFrame is an anti-pattern. It can crowd useful data, increase GC and make lineage harder to reason about. Another mistake is calling cache without any action

and claiming data is already stored. Cache is lazy too.

Retrieval prompt

A cached DataFrame is 60 GB but executors have limited memory. Explain how spill/eviction can reduce benefit and why MEMORY_ONLY thinking from old tutorials may not fit the default DataFrame cache behavior.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Repeated-action/caching context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: DataFrame.cache Persist a DataFrame for reuse [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.cache.html>

DE12-14 | Parquet I/O and partition pruning

Core recall

Choose partition layout from stable access patterns and volume, keep file sizes reasonable, and verify pruning/projection in the actual plan. Logical output checks stay identical regardless of physical layout.

Most dangerous wrong turn

Partitioning by high-cardinality order_id would create tiny directories/files and metadata overhead. Another error is thinking where(year(order_date)==2026) always prunes a directory partition exactly like a direct partition-column predicate; verify the plan. Do not infer pruning from fast execution on 18 rows.

Retrieval prompt

The dominant query shifts from daily date slices to customer-history lookups. Explain why the old physical layout may no longer be ideal and why you should measure before repartitioning historical data.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Parquet read/write and partitioned data context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark Parquet data source Parquet read/write/schema/partition discovery behavior [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-data-sources-parquet.html>

DE12-15 | Built-in functions vs Python UDFs

Core recall

Prefer optimizer-visible built-ins when they express the rule clearly. Use UDFs for genuine gaps, with explicit types, null/error behavior, tests and plan/runtime evidence.

Most dangerous wrong turn

Do not contort a clear transformation into unreadable built-ins merely to avoid every UDF. Also do not define a UDF for simple tasks Spark already supports, such as uppercasing, date extraction or conditional columns. And never forget a return type/NULL behavior contract.

Retrieval prompt

A Python UDF sometimes throws on null input. Design a test and repair contract without silently replacing null with a business value that was never specified.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Built-in expression style and transformation examples [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: PySpark SQL functions Optimizer-visible built-in functions catalogue [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/functions.html>

DE12-16 | Driver safety: collect, toPandas and bounded inspection

Core recall

Keep large data distributed. Materialize on the driver only after a proven bound or when the API inherently returns a small scalar. Treat driver-memory assumptions as explicit contracts.

Most dangerous wrong turn

`df.limit(5).toPandas()` is bounded; `df.toPandas().head()` is not, because the full DataFrame reaches the driver before pandas takes the head. Likewise, writing `len(df.collect())` to count rows is far worse than `df.count()`. API order matters.

Retrieval prompt

You need to email a 50-row quality report. Explain a safe Spark path that proves the result is ≤ 50 rows before converting to pandas, and what you would do if the bound fails.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Driver vs distributed work context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: DataFrame.collect Returns all rows to the driver; use deliberately [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.collect.html>

DE12-17 | Testing and debugging Spark transformations

Core recall

Test transformations on tiny deterministic data, separate logical tests from performance tests, and save failed assertion evidence before repair. A plan refactor should not silently alter business invariants.

Most dangerous wrong turn

Do not assert incidental row order unless the transformation contract includes ordering. Do not use the same buggy production function to generate 'expected' data. And do not replace tests with one final row count: a fan-out can be followed by a filter that returns the expected count but wrong identities/totals.

Retrieval prompt

A DataFrame contains floating calculations. Explain when approximate tolerance is appropriate and why loosening tolerance cannot be used to hide a true business discrepancy.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Transformation refresher; testing task remains independent [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Testing PySpark `assertDataFrameEqual/assertSchemaEqual` and test-framework patterns [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/testing_pyspark.html

DE12-18 | Integrated PySpark pipeline and distributed-processing checkpoint

Core recall

A Spark pipeline is mastered when you can rebuild it on changed data, prove output with independent controls, interpret the physical plan/runtime evidence, and explain resource/driver/partition safety. Save the full evidence packet; M13 then adds real Delta/lakehouse transaction semantics.

Most dangerous wrong turn

Common failure modes are implicit schema, accidental Pending/Cancelled inclusion, join fan-out, blind repartition(1), unbounded collect/toPandas, caching one-use data, claiming AQE/skew improvements without runtime evidence, and reporting the 25,230 control from memory instead of recomputing it from the supplied source.

Retrieval prompt

Use `practice/assessment/practical/parcel_orders.csv` as a changed domain. Build a Spark summary by region for Completed parcels, prove total gross=6000 and completed rows=7, then perform a fresh retest on `clinic_jobs.csv` where Completed amount=3500 across 6 jobs. Do not reuse hard-coded region names or amounts from the order fixture.

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Review the flow, then close video and build independently [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark SQL performance tuning Plan, partition, join, cache and AQE controls used in final checkpoint [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>